

Quicksort

For additional reading and resources, you can refer to

<https://jeffe.cs.illinois.edu/teaching/algorithms/book/00-intro.pdf> and

<https://jeffe.cs.illinois.edu/teaching/algorithms/book/01-recursion.pdf>

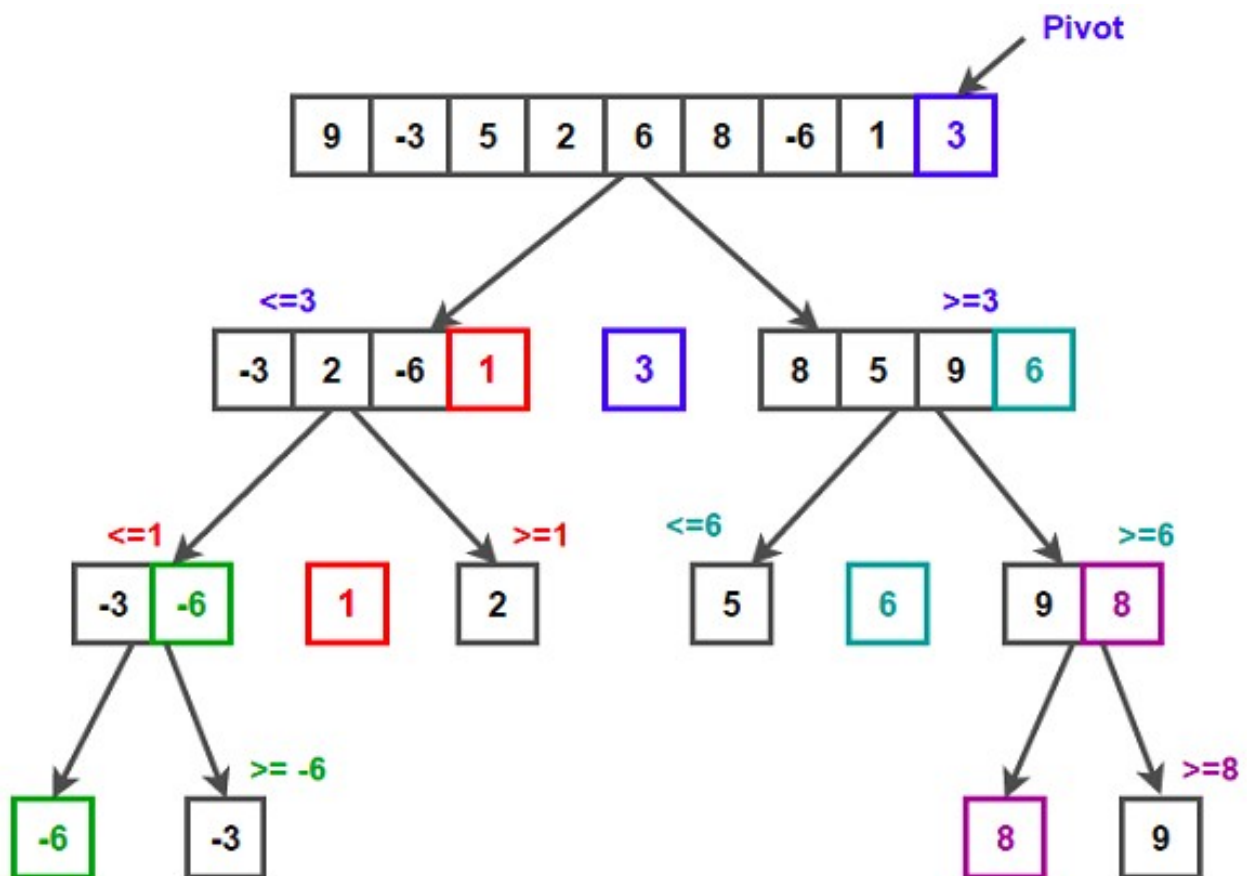
Earlier today, we learned about selection sort. Quicksort is another type of sort algorithm we will learn.

Quicksort

The high level algorithm for quicksort consists of three steps:

1. Choose a pivot element from the array
2. Partition the array into three subarrays: containing elements smaller than pivot, pivot itself, and elements larger than pivot
3. Recursively quicksort last and first subarrays

Below is a visual of what this looks like:



First, let's think about how to select a pivot element from the array. What are some ways we could select a pivot?

```
# How can we select a pivot?  
# last element, first element
```

Once we have selected a pivot, we need to actually partition elements based on the pivot. Think about what steps we would need to:

1. Figure out whether an element goes to the right or the left of the pivot
2. Moving elements from one side of the pivot to the other

```
# How can we figure out whether an element goes to the right or left  
of the pivot?
```

```
# How can we move elements from one side of the pivot to the other?
```

Let us think about it.

Quicksort Implementation

```
# Partition arr according to the pivot, and return the pivot position  
def partition(arr, left, right): # partition arr[left:right+1]  
    pivot = arr[right]  
    location = left  
    for i in range(left, right):  
        if (arr[i] < pivot):  
            arr[i], arr[location] = arr[location], arr[i]  
            location += 1  
    arr[location], arr[right] = arr[right], arr[location]  
    return location  
  
# Quicksort  
def quicksort(arr, left, right):  
    if (left >= right):  
        return  
    pivot_location = partition(arr, left, right)  
    quicksort(arr, left, pivot_location - 1)  
    quicksort(arr, pivot_location + 1, right)  
  
l_unsorted = [7, 1, 9, 20, 4, 6]  
print(l_unsorted)  
  
quicksort(l_unsorted, 0, len(l_unsorted) - 1)  
print(l_unsorted)  
  
[7, 1, 9, 20, 4, 6]  
[1, 4, 6, 7, 9, 20]
```

Quicksort Time Complexity

Quicksort time complexity can be dependent on your choice of pivot. Let's think through the worst case time complexity of quicksort. Assume we are picking the rightmost value in our array as our pivot. What are some cases where we would have the worst case time complexity?

```
# When do we have worst case quicksort?  
# When the array is already sorted, time complexity becomes  $O(n^2)$ 
```

What is quicksort time complexity? Let's take a look at the code we wrote and see if we can figure it out.

```
# Quicksort function code  
# Partition
```

However, by picking the pivot in a more clever way, the time complexity of the quicksort becomes $O(n \log n)$. We will not cover it in this class.